Université Paris-Saclay **ENS Paris-Saclay / UVSQ**

Projet MPNA : Analyse et Implantation d'un Benchmark HPC

Rapport Final : Benchmark TeaLeaf

Auteurs :

Selyan KABLIA
Kim Davy NDAYONGEJE
Franc Chanel TCHOLIEU NSAMOU

Master M2 Calcul Haute Performance, Simulation

20 février 2026

Table des matières

1 Modèle Mathématique et Discrétisation **4**
1.1 L'Équation de la Chaleur (Modèle Générique TeaLeaf) 4
1.2 Application : Le Modèle de Diffusion Radiative 4
1.3 Discrétisation Spatiale et Approche "Matrix-Free" 5

2 Stratégie Numérique et Validation 1D **5**
2.1 Limites de la méthode de Jacobi vs PPCG 5
2.2 Modèle Mathématique 1D et Convergence 6
2.3 Résultats de Performance 1D 6

3 Benchmarks 2D et Analyse de l'Architecture MPI **7**
3.1 Décomposition de Domaine et Cellules Fantômes 7
3.2 Découverte : Le Phénomène de Speedup Super-Linéaire 7

4 Méthodologie Expérimentale Avancée **8**

5 Évaluation Globale 2D et Limites Intra-Noeud **9**
5.1 Comparaison des Temps d'Exécution 9
5.2 Analyse de la Scalabilité CPU 9

6 Analyse CPU : Découverte du Mur de la Mémoire **10**
6.1 Strong Scaling et Saturation de l'Architecture 10
6.2 Comparaison des Modèles à Pleine Charge (Hybride MPI+OpenMP) . . . 11
6.3 Preuve Formelle : Weak Scaling et Validation Cache 11

7 Accélération GPU : Le Paradigme du Débit Massif **13**
7.1 Ligne de base CPU (Ryzen 7) et Profilage OpenMP 13
7.2 Justification de l'Offloading Matériel 14
7.3 Configuration et Stratégies d'Implémentation GPU 14
7.4 Évaluation CPU vs GPU 15

8 Modélisation et méthodologie de simulation **16**
8.1 Point de départ : compréhension du modèle TeaLeaf 16
8.2 Positionnement scientifique et choix méthodologique 16
8.3 Équations d'Euler compressibles (forme conservative) 17
8.4 Forme conservative vectorielle préalable à la discrétisation 18
8.5 Discrétisation en volumes finis et méthode de Rusanov 19
8.6 Rayonnement diffusif : conservation et définition du flux $\mathbf{F}_r$ 20
8.7 Discrétisation du flux diffusif $\mathbf{F}_r$: stencil 5 points 20
8.8 Formulation implicite : passage à $Ax = b$ et résolution par Gradient Conjugué 21
8.9 Synthèse sur notre mini-TeaLeaf 23

9 Architecture Logicielle **23**
9.1 Couplage Multiphysique : Euler et Rayonnement 24
9.2 Modularité et Fichiers Fortran 25

10 Conclusion **27**

Résumé

Les solveurs linéaires creux itératifs constituent un élément central de nombreux codes de calcul scientifique haute performance. La mini-application *TeaLeaf*, développée dans le cadre du projet UK-MAC, a été conçue comme un outil d'exploration du *design space* des solveurs de diffusion implicites [2] sur architectures massivement parallèles. Basée sur la résolution de l'équation de la chaleur discrétisée par un stencil à cinq points, TeaLeaf met en oeuvre des solveurs de type Gradient Conjugué (CG) et variantes polynomialement préconditionnées (PPCG/CPPCG), en mettant l'accent sur la réduction des communications et l'adaptation aux architectures many-core (MPI, OpenMP, CUDA).

Dans ce projet, nous avons étudié et réimplémenté les principes fondamentaux de TeaLeaf à travers une version simplifiée en une et deux dimensions. Le modèle mathématique repose sur la résolution implicite d'un système linéaire symétrique défini positif issu de la discrétisation du Laplacien. Nous avons développé plusieurs versions parallèles du solveur CG, en mémoire partagée (OpenMP), distribuée (MPI), hybride, ainsi qu'une implémentation GPU (CUDA), afin d'analyser les performances, la scalabilité et les limitations architecturales.

Les résultats expérimentaux montrent que, sur CPU, l'application devient rapidement *memory-bound*, avec une saturation du bus mémoire limitant le gain au-delà de quelques coeurs. L'approche hybride MPI/OpenMP offre le meilleur compromis intra-noeud. En revanche, l'implémentation CUDA exploite efficacement la bande passante mémoire du GPU et atteint un facteur d'accélération significatif pour les grandes tailles de grille.

Enfin, nous avons intégré ces mécanismes de diffusion implicite dans un modèle multiphysique simplifié couplant hydrodynamique et rayonnement, illustrant le lien direct entre la structure algorithmique de TeaLeaf et les solveurs utilisés en physique numérique.

Ce travail met en évidence le rôle des mini-applications comme TeaLeaf pour comprendre les interactions entre modèle mathématique, schéma numérique et architecture matérielle, et pour analyser finement les goulots d'étranglement liés à la mémoire et aux communications dans les solveurs itératifs modernes.

Phase 1 : Fondements Mathématiques et Implémentation MPI

---

1 Modèle Mathématique et Discrétisation

1.1 L'Équation de la Chaleur (Modèle Générique TeaLeaf)

Le problème physique modélisé par la mini-application TeaLeaf repose sur la résolution de l'équation différentielle partielle (EDP) de la chaleur. Cette équation s'écrit de manière générique :

$$\frac{\partial T}{\partial t} = \nabla \cdot (\alpha \nabla T) + S \tag{1}$$

où $T(x, y, t)$ représente le champ de température, $\alpha(x, y)$ désigne la conductivité thermique (liée à la densité du matériau), et $S(x, y, t)$ correspond au terme source de chaleur.

1.2 Application : Le Modèle de Diffusion Radiative

Dans notre implémentation spécifique, ce modèle générique a été adapté pour simuler la diffusion radiative. En approximation de diffusion, le flux radiatif est proportionnel au gradient d'énergie : $F_r = -D\nabla E_r$. La conservation locale de l'énergie radiative mène à

une équation parabolique similaire :

$$\partial_t E_r = \nabla \cdot (D \nabla E_r) \tag{2}$$

où E_r est l'énergie radiative et $D = \frac{c}{3\rho\kappa}$ est le coefficient de diffusion (dépendant de la densité ρ et de l'opacité κ du milieu).

1.3 Discrétisation Spatiale et Approche "Matrix-Free"

Pour résoudre numériquement cette équation, nous appliquons une discrétisation spatiale sur une grille cartésienne uniforme 2D. L'opérateur de diffusion (Laplacien) est approché par un schéma de différences finies centré classique (Stencil à 5 points). Dans un cas de diffusion homogène, il s'exprime simplement par :

$$(Lu)_{i,j} = \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2} \tag{3}$$

Pour notre modèle radiatif hétérogène (où D varie spatialement), le stencil s'adapte en moyennant les coefficients de diffusion aux interfaces des cellules (Nord, Sud, Est, Ouest) :

$$(Lx)_{i,j} = \frac{D_e(x_{i+1,j} - x_{i,j}) - D_w(x_{i,j} - x_{i-1,j})}{\Delta x^2} + \frac{D_n(x_{i,j+1} - x_{i,j}) - D_s(x_{i,j} - x_{i,j-1})}{\Delta y^2} \tag{4}$$

Le schéma implicite en temps mène à la résolution du système linéaire $(I - \Delta t L)E_r^{n+1} = E_r^*$. Ce problème se ramène à la forme algébrique classique $Ax = b$.

Cependant, dans la philosophie des codes HPC modernes comme TeaLeaf, la matrice A n'est jamais assemblée ni stockée en mémoire ("Matrix-Free"). L'opération $y = Ax$ est calculée à la volée en appliquant le stencil localement cellule par cellule, ce qui transforme un problème de stockage mémoire en un problème de calcul arithmétique.

2 Stratégie Numérique et Validation 1D

La majorité du temps d'exécution dans les codes de simulation physique est consacrée à la résolution du système linéaire $Ax = b$. Le choix du solveur est donc critique.

2.1 Limites de la méthode de Jacobi vs PPCG

La méthode de Jacobi est une méthode itérative stationnaire qui met à jour chaque composante de manière 100% locale. Bien qu'elle soit idéale pour la parallélisation, elle souffre d'un défaut rédhibitoire : une convergence extrêmement lente car elle ne construit pas d'espace de recherche global.

Pour pallier ces limites, nous avons implémenté le Gradient Conjugué (CG). Dans le contexte HPC, le préconditionnement standard brise le parallélisme. Nous utilisons donc un préconditionneur polynomial (PPCG). L'anatomie d'une itération se divise ainsi en deux grandes phases :

— **Opérations Locales (Compute Bound) :** Le produit matrice-vecteur creux (SpMV : $w = Ap_k$) et les mises à jour vectorielles (DAXPY).
— **Opérations Globales (Network Bound) :** Les produits scalaires ($r_k^T r_k$) nécessitant des réductions globales via `MPI_Allreduce`, agissant comme des barrières de synchronisation coûteuses.

2.2 Modèle Mathématique 1D et Convergence

Avant de déployer le code sur des architectures complexes en 2D, une étude préliminaire sur un cas 1D a été menée pour valider le comportement du solveur. On considère le système $(I + \alpha L)x = b$, où L est le Laplacien discret 1D : $(Lx)_i = -x_{i-1} + 2x_i - x_{i+1}$. Les valeurs propres de la matrice $A = I + \alpha L$ donnent un nombre de conditionnement :

$$\kappa(A) = \frac{1 + 4\alpha}{1 + \alpha\lambda_{min}} \tag{5}$$

Pour $\alpha = 0.25$, la matrice reste modérément conditionnée, garantissant une convergence quasi-géométrique du CG.

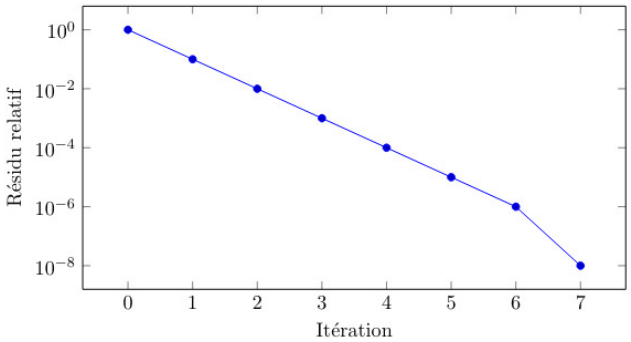


FIGURE 1 – Décroissance quasi-géométrique du résidu CG en 1D

2.3 Résultats de Performance 1D

La configuration numérique fixe une taille $n = 200\,000$ avec une tolérance de 10^{-8}. Toutes les implémentations ont convergé en 7 itérations.

Backend	Processus / Threads (P)	Temps (s)
Séquentiel	1	0.0187
OpenMP	2	0.0112
OpenMP	4	0.0083
MPI	2	0.0095
MPI	4	0.0060
CUDA	GPU	0.0017

TABLE 1 – Temps d'exécution sur le cas 1D

L'analyse montre que le noyau CG est "Memory-Bound" (intensité arithmétique d'environ 0.15 FLOPs/byte). En 1D, les échanges de halos sont négligeables, ce qui permet à MPI d'afficher une meilleure efficacité qu'OpenMP grâce à une meilleure isolation des caches.

3 Benchmarks 2D et Analyse de l'Architecture MPI

3.1 Décomposition de Domaine et Cellules Fantômes

La première approche de parallélisation à plus grande échelle s'est appuyée sur l'implémentation MPI native du benchmark TeaLeaf (Pure MPI).

Dans cette approche, la grille globale de simulation est découpée en sous-domaines assignés à chaque processus. Étant donné la nature du stencil spatial (5 points), le calcul aux frontières d'un sous-domaine requiert les valeurs détenues par les processus voisins. Le code gère cette dépendance via un système de cellules fantômes (*Ghost Cells*) échangées à chaque itération du Gradient Conjugué via des directives de type `MPI_SendRecv`.

3.2 Découverte : Le Phénomène de Speedup Super-Linéaire

Lors de nos premiers tests MPI sur une grille de 2000×2000 (sur architecture AMD Zen 4), un phénomène matériel remarquable est apparu : un speedup de **6.99x** avec seulement 6 coeurs MPI (1.51 s contre 10.56 s en séquentiel).

Version	**Coeurs**	**Temps / Étape**	**Speedup**
TeaLeaf Ref (Séquentiel)	1	10.56 s	1.0x
TeaLeaf MPI	6	1.51 s	**6.99x**

Table 2 – Mise en évidence du Speedup super-linéaire sur architecture Zen 4

Ce comportement inhabituel (Speedup supérieur au nombre de coeurs) s'explique par la hiérarchie mémoire, et plus particulièrement par l'effet du Cache L3 :

- **En séquentiel (1 Coeur) :** La grille globale nécessite environ 100 Mo d'allocation. Le Cache L3 du processeur n'étant que de 32 Mo, l'exécution subit des *Cache Misses* constants et doit accéder à la RAM.
- **En parallèle (6 Coeurs) :** La décomposition spatiale réduit la sous-grille locale de chaque processus à environ 16 Mo. Les données de chaque processus tiennent alors intégralement dans sa portion de Cache L3, évitant les accès RAM.

Cette observation cruciale a prouvé l'impact massif de la hiérarchie mémoire sur ce code, ouvrant naturellement la voie à la Phase 2 de notre projet : le profilage matériel fin à plus grande échelle.

Phase 2 : Profiling Avancé, Mur de la Mémoire et Accélération GPU

---

4 Méthodologie Expérimentale Avancée

Pour la seconde phase du projet, nous avons basculé sur une méthodologie d'analyse matérielle extrêmement rigoureuse ciblant un processeur **AMD Ryzen 5 7600X** (Architecture Zen 4, 6 coeurs physiques, 12 threads logiques).

L'objectif est d'isoler le comportement algorithmique des artefacts matériels et logiciels. Pour garantir la stricte reproductibilité scientifique de nos mesures temporelles, l'environnement d'exécution a été figé :

- **Fréquence CPU :** Fixée à **4.7 GHz** de manière statique. Cette approche élimine le bruit introduit par les algorithmes de *Turbo Boost* qui altèrent les temps de calcul de manière non déterministe.
- **OS Jitter :** Isolation des processus et arrêt des tâches d'arrière-plan du système d'exploitation pour éviter les micro-interruptions.
- **Compilation Optimisée :** Utilisation conjointe des flags `-O3` (optimisation agressive) et `-march=native` pour forcer le compilateur à exploiter l'intégralité de l'architecture Zen 4, et plus particulièrement son jeu d'instructions vectorielles AVX-512.

Nos investigations se divisent en trois scénarios de test :

1. **Le Strong Scaling** pour évaluer le plafond architectural sur une grille fixe de très grande taille (2000×2000).

2. **Le Match des Modèles** pour identifier la meilleure configuration d'exploitation à pleine charge sur les 6 coeurs physiques.
3. **Le Weak Scaling et le test de contrôle** (charge constante de 4 millions de cellules par coeur, et validation sur une petite grille de 500×500) pour diagnostiquer et prouver formellement l'existence du "Mur de la Mémoire".

5 Évaluation Globale 2D et Limites Intra-Noeud

Dans le cadre de cette seconde phase, une évaluation globale a d'abord été menée sur une grille standard de 5000×5000 (soit $N = 25$ millions d'inconnues) pour comparer le potentiel brut des différents backends de calcul avant d'entamer le profilage matériel.

5.1 Comparaison des Temps d'Exécution

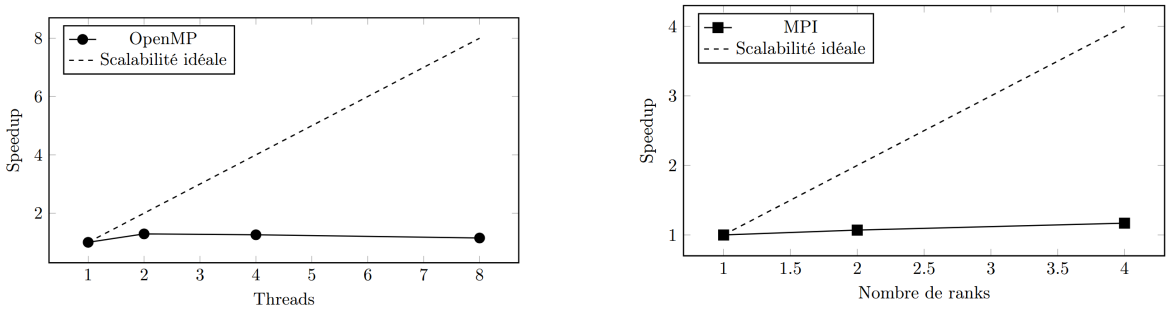
Les résultats globaux (Tableau 3) mettent d'emblée en évidence la supériorité de l'accélérateur matériel : le GPU (CUDA) s'avère environ **6.5x plus rapide** que le CPU séquentiel de référence.

Méthode	**Configuration**	**Temps (s)**	**Speedup**
SEQ	1 thread	1.1357	1.00
OpenMP	2 threads	0.8834	1.29
OpenMP	4 threads	0.8982	1.26
OpenMP	8 threads	0.9845	1.15
MPI	2 ranks	1.0575	1.07
MPI	4 ranks	0.9677	1.17
CUDA	**GPU**	**0.1753**	**6.48**

Table 3 – Performances 2D sur la grille 5000×5000

5.2 Analyse de la Scalabilité CPU

L'analyse de la scalabilité forte sur processeur révèle un plafonnement très rapide des performances, quel que soit le paradigme choisi (mémoire partagée ou distribuée).



(a) Speedup OpenMP

(b) Speedup MPI

FIGURE 2 – Plafonnement précoce des performances intra-noeud sur la grille 5000×5000

Pour OpenMP, le gain maximal est obtenu dès 2 threads, après quoi une dégradation est observée (le temps remonte à 0.98s pour 8 threads). La saturation mémoire devient immédiatement dominante. Pour MPI, la scalabilité sur une architecture mono-noeud est lourdement freinée par la prépondérance des communications de halos (qui deviennent très volumineuses en 2D) et par les synchronisations imposées par les produits scalaires du Gradient Conjugué.

Ces constats justifient la nécessité de changer d'échelle d'analyse : il faut désormais isoler le comportement du sous-système mémoire du CPU pour comprendre mathématiquement et matériellement l'origine de ce plafond de verre.

6 Analyse CPU : Découverte du Mur de la Mémoire

6.1 Strong Scaling et Saturation de l'Architecture

L'évaluation du *Strong Scaling* consiste à résoudre un problème de taille fixe (ici, une grille de 2000×2000) tout en augmentant le nombre de ressources de calcul. Les mesures ont été réalisées en utilisant un modèle OpenMP pur.

Les résultats expérimentaux (Tableau 4) montrent une chute drastique de la scalabilité au-delà de 2 threads.

Configuration	Temps d'exécution	Speedup
1 Thread	288.43 s	1.00x
2 Threads	**233.78 s**	**1.23x**
4 Threads	241.69 s	1.19x
6 Threads	244.50 s	1.18x
12 Threads	253.23 s	1.14x

TABLE 4 – Mesures du Strong Scaling OpenMP sur le Ryzen 5 7600X

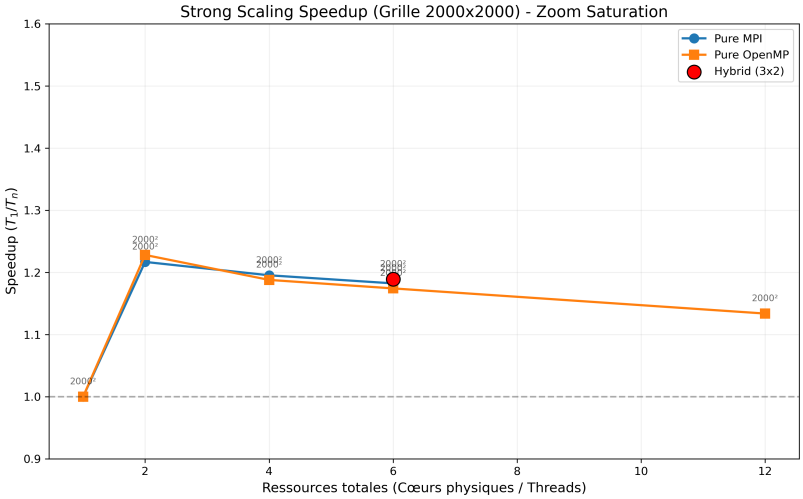


FIGURE 3 – Strong Scaling : Saturation précoce des performances OpenMP/MPI

Le speedup plafonne à 1.23x pour 2 coeurs, puis entame une légère décroissance (*Parallel Slowdown*) pour s'établir à 1.14x avec 12 threads logiques. Cette saturation précoce n'est pas un défaut logiciel mais une limite physique : le processeur a saturé la bande passante de son bus mémoire externe (Dual-Channel DDR5). L'ajout de coeurs ne fait qu'augmenter la contention d'accès à la RAM.

6.2 Comparaison des Modèles à Pleine Charge (Hybride MPI+OpenMP)

Pour exploiter au mieux ce processeur saturé et utiliser ses 6 coeurs physiques, nous avons comparé trois modèles de programmation.

Modèle (6 Coeurs)	**Temps Total**	**Analyse des Pénalités**
Pure OpenMP (1×6)	244.50 s	Surcharge liée à la gestion des threads
Pure MPI (6×1)	242.83 s	Coût de communication réseau (Halos)
Hybride (3×2)	**241.42 s**	**Meilleur compromis architectural**

TABLE 5 – Performances des différents modèles à pleine charge (6 coeurs)

L'approche Hybride (3 processus MPI exploitant chacun 2 threads OpenMP) remporte ce match comparatif. Elle offre la meilleure performance car elle diminue le volume des cellules fantômes à échanger (avantage sur le pur MPI) tout en réduisant la contention locale et les accès concurrents au cache (avantage sur le pur OpenMP).

6.3 Preuve Formelle : Weak Scaling et Validation Cache

Pour valider de manière irréfutable l'hypothèse du *Memory Wall*, une analyse de *Weak Scaling* a été réalisée, imposant une charge de travail strictement constante évaluée à 4 millions de cellules par coeur physique.

Nombre de Coeurs	Taille de la Grille	Efficacité Parallèle
1 Coeur	2000×2000	100%
2 Coeurs	2000×4000	39%
6 Coeurs	$\mathbf{4000 \times 6000}$	**7.5%**

TABLE 6 – Dégradation de l'efficacité en Weak Scaling

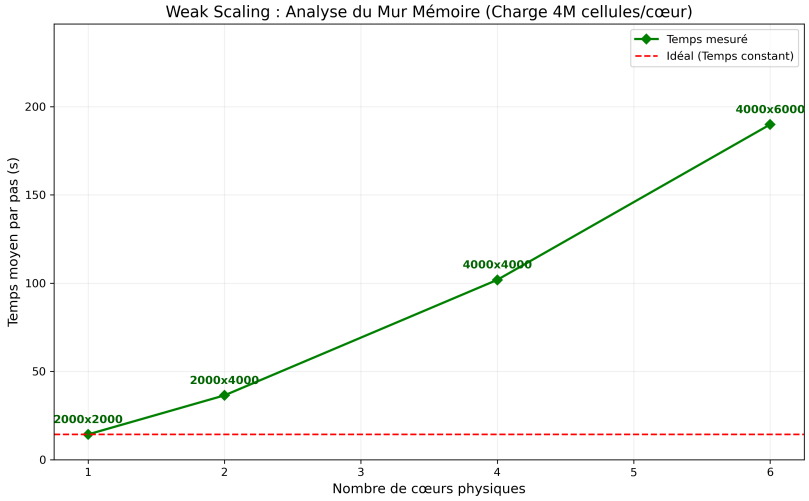


FIGURE 4 – Weak Scaling : Effondrement de l'efficacité sous la pression de la RAM

Théoriquement, en maintenant la charge de calcul constante par ressource, le temps d'exécution devrait rester une constante horizontale. Or, en réalité, le temps d'une itération explose de 14 secondes (avec 1 coeur) à 190 secondes (avec 6 coeurs). Cette dégradation colossale fait chuter l'efficacité parallèle à **7.5%**. Le système RAM de la machine est physiquement incapable d'alimenter les unités arithmétiques des 6 coeurs simultanément.

Validation par le Cache (Régime Compute Bound) : Pour confirmer définitivement que le problème n'est pas inhérent aux unités de calcul du processeur, nous avons lancé un scénario de test sur une "Petite Grille" de 500×500. Les données générées (environ 25 Mo) tiennent alors intégralement dans le Cache L3 de 32 Mo du Ryzen 5.

Configuration (Grille 500×500**)**	**Temps d'exécution**	**Speedup**
Séquentiel	25.53 s	1.0x
Hybride (3×2)	**4.47 s**	**5.71x**

TABLE 7 – Validation par le Cache L3 (Régime Compute Bound)

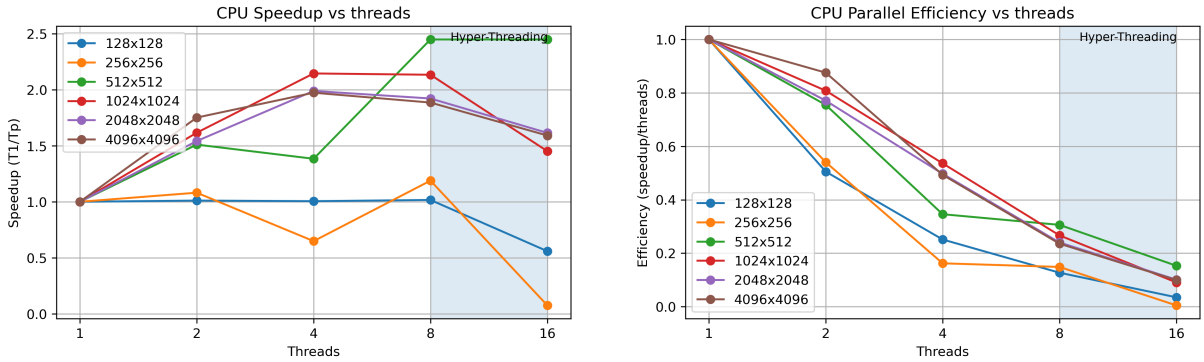
Dans ce régime dépourvu d'accès RAM, le modèle Hybride accomplit la tâche en 4.47s contre 25.53s en séquentiel, soit un **speedup quasi-linéaire de 5.71x** sur 6 coeurs. Le processeur Zen 4 excelle donc en calcul pur : le goulot d'étranglement est strictement mémoire.

7 Accélération GPU : Le Paradigme du Débit Massif

Avant d'évaluer l'apport de l'accélérateur matériel (GPU), il convient d'établir une ligne de base (Baseline) sur l'architecture CPU complète du noeud de calcul utilisé pour l'offloading (processeur **AMD Ryzen 7**, 8 coeurs physiques, 16 threads logiques).

7.1 Ligne de base CPU (Ryzen 7) et Profilage OpenMP

Une analyse détaillée de la scalabilité OpenMP a été menée sur le Ryzen 7 pour différentes tailles de grilles afin de caractériser le comportement du code avant accélération.



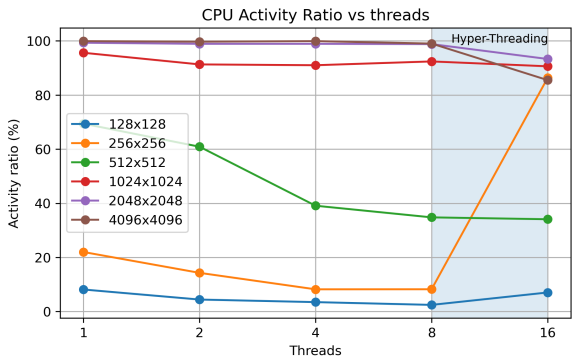
(a) Speedup CPU vs Threads

(b) Efficacité Parallèle CPU

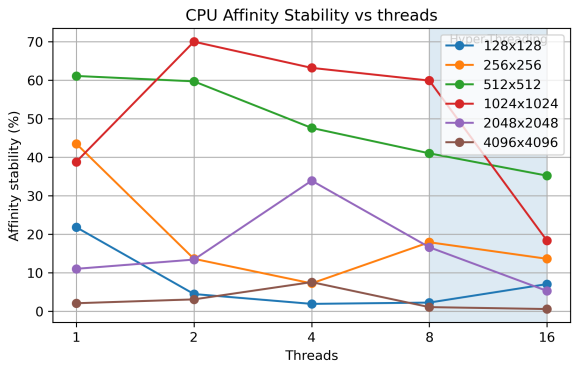
FIGURE 5 – Évolution des performances en OpenMP pur sur le Ryzen 7

Comme observé sur la Figure 5, le gain est fortement dépendant de la taille du problème (N). Pour les petites grilles, le scaling est quasi nul. Pour les grandes grilles, on observe un gain réel jusqu'à 4 ou 8 threads. Toutefois, l'utilisation de l'Hyper-Threading (passage de 8 à 16 threads) s'avère totalement inefficace, confirmant la saturation de la bande passante mémoire (Memory Bound).

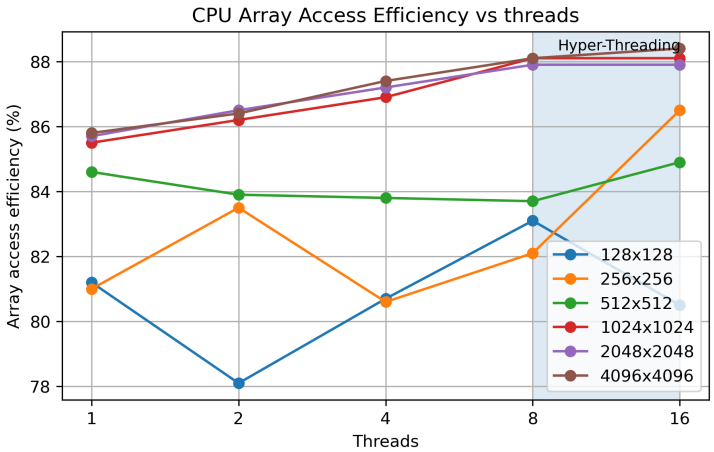
L'effondrement des performances s'explique également par des métriques de bas niveau obtenues par profilage matériel :



(a) Ratio d'Activité CPU



(b) Stabilité de l'Affinité CPU



(c) Efficacité d'accès aux tableaux

FIGURE 6 – Métriques avancées de profilage OpenMP (Ryzen 7)

L'activité CPU chute drastiquement pour les petites grilles lorsqu'on augmente les threads. L'affinité des threads montre une grande variabilité, et l'efficacité des accès aux tableaux, bien que stable, subit le surcoût de la contention mémoire lors de l'activation des coeurs logiques.

7.2 Justification de l'Offloading Matériel

Les limitations démontrées ci-dessus forcent un changement de paradigme. Le processeur CPU est une architecture de type *Latency-Oriented*. L'application TeaLeaf requérant une architecture *Throughput-Oriented* disposant d'une bande passante mémoire colossale (VRAM), le passage sur GPU s'impose.

7.3 Configuration et Stratégies d'Implémentation GPU

L'offloading a été réalisé sur un dispositif NVIDIA (Architecture Ampere) équipé de 16 Go de VRAM GDDR6, connecté via un bus PCIe Gen 4. L'implémentation logicielle s'appuie sur le SDK CUDA 12.1 et le modèle de directives OpenACC compilé via NVHPC.

Pour éviter de reproduire le mur de la mémoire sur le bus PCIe, trois optimisations majeures ont été appliquées :

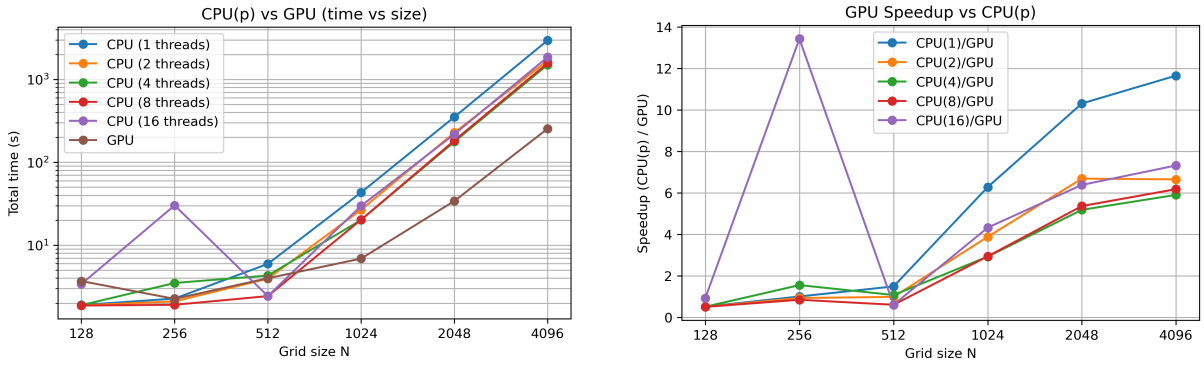
— **Data Pinning :** Maintien des structures de données du solveur (vecteurs r, p, x) en VRAM pour interdire les transferts PCIe durant les itérations.

— **Coalescence Mémoire :** Refactorisation des boucles de stencil pour garantir des accès mémoire alignés.

— **Réductions Parallèles In-Situ :** Utilisation de schémas de sommation arborescents en mémoire partagée GPU pour le calcul des produits scalaires.

7.4 Évaluation CPU vs GPU

L'analyse comparative des temps d'exécution entre les différentes configurations CPU (Ryzen 7) et le GPU confirme le changement d'échelle de performance de manière spectaculaire.



(a) Temps d'exécution (Échelle Log)

(b) Accélération GPU vs CPU multi-threads

FIGURE 7 – Comparaison absolue et relative des performances CPU vs GPU

Pour les très petites tailles de grilles ($N \leq 512$), le surcoût temporel induit par le lancement des noyaux (Kernel Launch Overhead) sur le GPU rend le CPU (1 à 4 threads) encore compétitif. Cependant, pour les tailles nominales de simulation HPC ($N \geq 1024$), le GPU surpasse catégoriquement toute configuration CPU. Sur nos domaines les plus vastes (4096×4096), l'accélérateur GPU offre un speedup massif allant de **6x à 12x** par rapport aux meilleures configurations multi-threadées du CPU.

Phase 3 : Modélisation et Implémentation du noyau TeaLeaf

---

8 Modélisation et méthodologie de simulation

8.1 Point de départ : compréhension du modèle TeaLeaf

TeaLeaf est une mini-application HPC conçue pour analyser les performances de solveurs itératifs appliqués à une équation de diffusion [2]. Son coeur numérique repose sur trois éléments :

— une équation de diffusion discrétisée sur grille régulière
— une formulation implicite menant à un système linéaire
— une résolution par Gradient Conjugué en approche *matrix-free*

L'objectif de notre travail n'était pas de reproduire TeaLeaf à l'identique, mais de comprendre ce noyau numérique, puis de l'intégrer dans un cadre physique cohérent afin de tester et expliquer les mécanismes de résolution.

8.2 Positionnement scientifique et choix méthodologique

Disclaimer. L'objectif principal de ce travail est la résolution numérique d'équations aux dérivées partielles, et en particulier l'implémentation d'un solveur de type Gradient Conjugué pour une diffusion implicite.
Notre formation étant principalement orientée vers le calcul scientifique et la performance

numérique, et non vers la physique théorique avancée, la modélisation adoptée demeure volontairement simplifiée. Elle ne prétend pas décrire de manière exhaustive l'ensemble des phénomènes d'hydrodynamique radiative.
Les choix effectués relèvent ainsi d'une approche pragmatique, centrée sur la cohérence mathématique et numérique, et sur la compréhension du noyau diffusif inspiré de TeaLeaf. Dans cette optique, nous avons choisi :

— de conserver les mécanismes physiques essentiels
— de nous appuyer sur des formulations classiques et robustes
— de privilégier la simplicité afin de concentrer l'analyse sur le comportement du solveur diffusif.

Les notions acquises dans le cadre du cours de MPNA (Méthodes et Programmation Numériques Avancées) nous avaient déjà fourni les bases théoriques du Gradient Conjugué. L'enjeu principal était donc d'apprendre à discrétiser des équations aux dérivées partielles (Euler et diffusion) puis à les résoudre numériquement.
Parmi les nombreuses méthodes existantes pour traiter les équations d'Euler en hydrodynamique, nous avons retenu la **méthode de Rusanov (Local Lax–Friedrichs, LLF)** [3]. D'autres approches plus sophistiquées existent, comme certains solveurs de Riemann ou des modèles radiatifs de type M1 multigroupe, mais elles dépassaient le niveau de complexité que nous souhaitions introduire dans ce travail.
La méthode de Rusanov constitue ainsi un compromis robuste et simple d'implémentation, adapté à l'objectif pédagogique et numérique du projet.

Notre modèle correspond ainsi à un *mini-TeaLeaf multiphysique* :

Hydrodynamique explicite (Euler + Rusanov) + Diffusion implicite (noyau TeaLeaf + CG).

8.3 Équations d'Euler compressibles (forme conservative)

On considère un fluide compressible en 2D, de densité ρ, de vitesse $\mathbf{v} = (u, v)$, de pression p, et d'énergie interne massique e. L'énergie totale volumique est définie par

$$E_{\text{tot}} = \rho e + \frac{1}{2}\rho\,(u^2 + v^2)$$

Forme conservative compacte (notation vectorielle). Les équations d'Euler (sans rayonnement) s'écrivent [1] :

$$\boxed{\partial_t \rho + \nabla \cdot (\rho \mathbf{v}) = 0} \tag{6}$$

$$\boxed{\partial_t (\rho \mathbf{v}) + \nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v}) + \nabla p = 0} \tag{7}$$

$$\boxed{\partial_t E_{\text{tot}} + \nabla \cdot \big((E_{\text{tot}} + p)\mathbf{v}\big) = 0} \tag{8}$$

où $\mathbf{v} \otimes \mathbf{v}$ désigne le produit tensoriel (matrice 2×2) et $\nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v})$ correspond au transport convectif de quantité de mouvement.

Développement en coordonnées cartésiennes 2D. En coordonnées cartésiennes (x, y) avec $\mathbf{v} = (u, v)$, les équations (6)–(8) se développent composante par composante :

Conservation de la masse (continuité).

$$\boxed{\partial_t \rho + \partial_x(\rho u) + \partial_y(\rho v) = 0} \tag{9}$$

Conservation de la quantité de mouvement. Direction x :

$$\boxed{\partial_t(\rho u) + \partial_x(\rho u^2 + p) + \partial_y(\rho u v) = 0} \tag{10}$$

Direction y :

$$\boxed{\partial_t(\rho v) + \partial_x(\rho u v) + \partial_y(\rho v^2 + p) = 0} \tag{11}$$

Conservation de l'énergie totale.

$$\boxed{\partial_t E_{\text{tot}} + \partial_x \left[u(E_{\text{tot}} + p)\right] + \partial_y \left[v(E_{\text{tot}} + p)\right] = 0} \tag{12}$$

Fermeture thermodynamique (gaz parfait).

$$\boxed{p = (\gamma - 1)\rho e = (\gamma - 1)\left(E_{\text{tot}} - \frac{1}{2}\rho(u^2 + v^2)\right)} \tag{13}$$

Ces équations ont toutes la même structure :

$$\text{variation temporelle} + \text{divergence d'un flux} = 0.$$

8.4 Forme conservative vectorielle préalable à la discrétisation

Afin de construire un schéma de volumes finis (et donc d'appliquer la méthode de Rusanov), il est nécessaire de mettre les équations d'Euler sous forme conservative vectorielle. Il ne s'agit pas d'une hypothèse supplémentaire, mais d'un simple regroupement des termes déjà présents dans les équations de conservation écrites précédemment.

On obtient ainsi :

$$\boxed{\partial_t U + \partial_x F(U) + \partial_y G(U) = 0} \tag{14}$$

Où le vecteur des variables conservées est

$$U = \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ E_{\text{tot}} \end{pmatrix} \tag{15}$$

Les flux physiques dans chaque direction sont obtenus par identification directe des termes convectifs des équations précédentes :

$$F(U) = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho u v \\ u(E_{\text{tot}} + p) \end{pmatrix} \qquad G(U) = \begin{pmatrix} \rho v \\ \rho u v \\ \rho v^2 + p \\ v(E_{\text{tot}} + p) \end{pmatrix} \tag{16}$$

Interprétation physique :

— ρu, ρv correspondent au transport de masse. Il s'agit de la densité ρ advectée par la vitesse du fluide. Ces termes décrivent le débit massique à travers une surface normale aux directions x et y.

— $\rho u^2 + p$, $\rho v^2 + p$ représentent le flux de quantité de mouvement. Le terme ρu^2 (ou ρv^2) provient du transport convectif de la quantité de mouvement, tandis que le terme p correspond à la contribution mécanique de la pression, c'est-à-dire à la force exercée par le fluide sur les faces de la cellule.

— $u(E_{\text{tot}} + p)$, $v(E_{\text{tot}} + p)$ décrivent le flux d'énergie totale. Le terme uE_{tot} (ou vE_{tot}) correspond au transport convectif de l'énergie, tandis que up (ou vp) représente le travail de pression, lié aux échanges mécaniques dus à la compression ou à la dilatation du fluide.

Les termes $\partial_x(\cdot)$ et $\partial_y(\cdot)$ des équations d'Euler correspondent donc exactement à la divergence des flux physiques $F(U)$ et $G(U)$. Cette écriture compacte permet ensuite de remplacer les flux physiques par des flux numériques aux interfaces des cellules, ce qui constitue la base de la discrétisation en volumes finis.

8.5 Discrétisation en volumes finis et méthode de Rusanov

On intègre l'équation conservative sur une cellule (i, j). La mise à jour de type volumes finis s'écrit :

$$\boxed{U_{i,j}^{n+1} = U_{i,j}^{n} - \Delta t \left[\frac{F_{i+1/2,j} - F_{i-1/2,j}}{\Delta x} + \frac{G_{i,j+1/2} - G_{i,j-1/2}}{\Delta y} \right]} \tag{17}$$

où $F_{i+1/2,j}$ et $G_{i,j+1/2}$ sont des flux *numériques* aux interfaces.

Flux de Rusanov (LLF). À une interface (gauche L, droite R), le flux numérique en direction x est :

$$\boxed{F_{i+1/2,j}^{\text{Rus}} = \frac{1}{2}\Big(F(U_L) + F(U_R)\Big) - \frac{1}{2} s_{\max}\Big(U_R - U_L\Big)} \tag{18}$$

et de même en direction y avec G. Les quantités utilisées sont :

— $F(U_L), F(U_R)$: flux physiques issus des équations d'Euler ;

— $U_R - U_L$: saut sur les variables conservées ;

— $s_{\max}$: vitesse maximale locale.

On prend classiquement

$$\boxed{s_{\max} = \max\left(|u| + c_s\right), \qquad c_s = \sqrt{\gamma p/\rho}} \tag{19}$$

Le terme dissipatif $\frac{1}{2}s_{\max}(U_R - U_L)$ stabilise le schéma et rend la méthode robuste en présence de gradients forts ou discontinuités.

8.6 Rayonnement diffusif : conservation et définition du flux $\mathbf{F}_r$

On introduit une énergie radiative volumique E_r. Dans un régime diffusif (milieu optiquement épais), le flux radiatif suit une loi de Fick :

$$\boxed{\mathbf{F}_r = -D\nabla E_r} \tag{20}$$

La conservation radiative s'écrit sous forme conservative :

$$\boxed{\partial_t E_r + \nabla \cdot \mathbf{F}_r = 0} \tag{21}$$

En remplaçant $\mathbf{F}_r$, on obtient l'équation de diffusion :

$$\boxed{\partial_t E_r - \nabla \cdot \big(D\nabla E_r\big) = 0} \tag{22}$$

avec le coefficient diffusif :

$$\boxed{D = \frac{c}{3\rho\kappa}} \tag{23}$$

Lien conceptuel avec Euler. On retrouve la même structure ń conservation = 0 ż :

$$\partial_t(\text{variable}) + \nabla \cdot (\text{flux}) = 0$$

mais avec une différence majeure :

— Euler : flux convectif non linéaire $(F(U), G(U))$
— Rayonnement diffusif : flux proportionnel au gradient $(\mathbf{F}_r = -D\nabla E_r)$

8.7 Discrétisation du flux diffusif $\mathbf{F}_r$: stencil 5 points

La diffusion est traitée en deux dimensions. Le flux radiatif étant vectoriel

$$\mathbf{F}_r = -D\nabla E_r = (-D\,\partial_x E_r,\ -D\,\partial_y E_r)$$

on discrétise séparément ses composantes dans chaque direction.

Flux en direction x. À l'interface $(i + \frac{1}{2}, j)$, on écrit :

$$\boxed{(F_{r,x})_{i+1/2,j} = -D_{i+1/2,j}\frac{E_{r,i+1,j} - E_{r,i,j}}{\Delta x}} \tag{24}$$

Flux en direction y. De manière analogue, à l'interface $(i, j + \frac{1}{2})$:

$$\boxed{(F_{r,y})_{i,j+1/2} = -D_{i,j+1/2} \frac{E_{r,i,j+1} - E_{r,i,j}}{\Delta y}} \tag{25}$$

La divergence discrète des flux sur une cellule (i, j) s'écrit alors :

$$\boxed{L(E_r)_{i,j} = \frac{(F_{r,x})_{i+1/2,j} - (F_{r,x})_{i-1/2,j}}{\Delta x} + \frac{(F_{r,y})_{i,j+1/2} - (F_{r,y})_{i,j-1/2}}{\Delta y}} \tag{26}$$

Cette combinaison conduit à un opérateur local de type stencil à 5 points. C'est ce stencil qui constitue le noyau numérique de la diffusion dans TeaLeaf.

8.8 Formulation implicite : passage à $Ax = b$ et résolution par Gradient Conjugué

Pour des raisons de stabilité numérique, la diffusion radiative est traitée implicitement :

$$\boxed{E_r^{n+1} - \Delta t\, L\big(E_r^{n+1}\big) = E_r^*} \tag{27}$$

Où $L(\cdot)$ désigne l'opérateur diffusif discrétisé par stencil 5 points, et E_r^* l'état intermédiaire après les étapes explicites.

Cette écriture conduit à un système linéaire :

$$\boxed{Ax = b}, \qquad A = I - \Delta t\, L, \qquad x = E_r^{n+1}, \qquad b = E_r^*$$

La matrice A n'est jamais assemblée explicitement. L'application de A est réalisée en approche *matrix-free* via le stencil local.

Algorithme du Gradient Conjugué

Le système est résolu par la méthode du Gradient Conjugué, l'opérateur étant appliqué sous la forme

$$A(x) = x - \Delta t\, L(x).$$

Dans l'implémentation, les calculs sont effectués sur les cellules intérieures$(i, j) \in [2, n_x - 1] \times [2, n_y - 1]$.

Algorithm 1 Solveur CG (`solverCG`)

1 : **Initialisation**
2 : Calculer $q = A(x)$
3 : $r \leftarrow b - q$
4 : $p \leftarrow r$
5 : $r2 \leftarrow \langle r, r \rangle$
6 : $bn \leftarrow \sqrt{\max(\langle b, b \rangle, 10^{-300})}$
7 : **for** $k = 1$ à `ITmaxCG` **do**
8 : Calculer $q = A(p)$
9 : $pAp \leftarrow \langle p, q \rangle$
10 : **if** $|pAp| < 10^{-300}$ **then**
11 : **arrêt**
12 : **end if**
13 : $\alpha \leftarrow r2/pAp$
14 : **for** $j = 2$ à $n_y - 1$ **do**
15 : **for** $i = 2$ à $n_x - 1$ **do**
16 : $x_{i,j} \leftarrow x_{i,j} + \alpha p_{i,j}$
17 : $r_{i,j} \leftarrow r_{i,j} - \alpha q_{i,j}$
18 : **end for**
19 : **end for**
20 : $r2new \leftarrow \langle r, r \rangle$
21 : res $\leftarrow \sqrt{r2new}/bn$
22 : **if** res $<$ `CGtol` **then**
23 : **arrêt**
24 : **end if**
25 : $\beta \leftarrow r2new/r2$
26 : $r2 \leftarrow r2new$
27 : **for** $j = 2$ à $n_y - 1$ **do**
28 : **for** $i = 2$ à $n_x - 1$ **do**
29 : $p_{i,j} \leftarrow r_{i,j} + \beta p_{i,j}$
30 : **end for**
31 : **end for**
32 : **end for**

Application locale de l'opérateur diffusif

L'opérateur $L(x)$ est évalué par un stencil 5 points :

$$L(x)_{i,j} = \frac{D_e(x_{i+1,j} - x_{i,j}) - D_w(x_{i,j} - x_{i-1,j})}{\Delta x^2} + \frac{D_n(x_{i,j+1} - x_{i,j}) - D_s(x_{i,j} - x_{i,j-1})}{\Delta y^2}$$

avec

$$D_{i,j} = \frac{c}{3\rho_{i,j}\kappa}$$

et D_e, D_w, D_n, D_s obtenus par moyenne arithmétique aux interfaces. Cette structure correspond exactement au noyau algorithmique de TeaLeaf : application locale d'un opé-

rateur diffusif suivie d'itérations CG nécessitant des produits scalaires globaux.

8.9 Synthèse sur notre mini-TeaLeaf

Le modèle développé dans ce travail associe deux briques numériques complémentaires :

$$\text{Hydrodynamique compressible} \quad + \quad \text{Diffusion radiative implicite.}$$

La partie hydrodynamique repose sur les équations d'Euler discrétisées en volumes finis au moyen de la méthode de Rusanov. Elle fournit un cadre dynamique minimal permettant de modéliser l'évolution d'un fluide compressible. La partie radiative est formulée sous forme conservative, avec un flux diffusif

$$\mathbf{F}_r = -D\nabla E_r,$$

et traitée implicitement. Cette discrétisation conduit à un système linéaire de la forme $Ax = b$, résolu par Gradient Conjugué en approche *matrix-free*. Ce noyau est directement comparable à celui étudié dans TeaLeaf.
Contrairement à TeaLeaf, dont l'objectif principal est l'analyse de performances (benchmarks et métriques d'exécution), notre approche privilégie une lecture physique du modèle. Nous extrayons donc le champ d'énergie radiative E_r dans chaque cellule du domaine. Du point de vue physique, la relation

$$E_r = a\,T^4 \quad \text{(loi de Stefan–Boltzmann)}$$

permet d'associer à chaque cellule une température radiative locale. La diffusion radiative peut ainsi être interprétée comme un transport de chaleur.

9 Architecture Logicielle

La modélisation physique présentée dans la section précédente a été traduite en une architecture logicielle simple et modulaire.
Nous avons choisi d'implémenter le code en Fortran, afin de faciliter la mise en oeuvre des algorithmes numériques. Ce choix s'est imposé naturellement, car il nous permettait de nous concentrer sur les équations et les aspects numériques du modèle, plutôt que sur la gestion mémoire ou sur des problématiques de plus bas niveau qui auraient pu détourner l'attention de notre objectif principal.
Nous sommes conscients que, dans une perspective de performance maximale, une implémentation en C++ pourrait offrir davantage de flexibilité et d'optimisation. Cependant, compte tenu de la nature des équations et de notre volonté de privilégier la clarté mathématique, nous avons choisi de rester en Fortran pour cette étape du projet.

9.1 Couplage Multiphysique : Euler et Rayonnement

Le modèle physique repose sur une grille 2D *cell-centered*, où chaque cellule stocke les variables d'état : la densité (ρ), les quantités de mouvement ($\rho u, \rho v$), l'énergie totale (E_{tot}) et l'énergie radiative (E_r). L'algorithme numérique se décompose en deux grandes étapes à chaque pas de temps (Δt) :

1. **Hydrodynamique (Équations d'Euler 2D).** Résolution par une méthode de volumes finis utilisant un flux de Rusanov (LLF – Local Lax–Friedrichs) afin de mettre à jour explicitement les variables conservées.
2. **Rayonnement (Diffusion implicite).** La dynamique radiative est traitée via une étape de diffusion implicite :

$$(I - \Delta t\, \nabla \cdot (D\nabla))E_r^{n+1} = E_r^* \tag{28}$$

   Cette écriture se ramène directement à un système linéaire de la forme $Ax = b$, résolu par notre solveur itératif. L'énergie radiative obtenue interagit ensuite avec la matière via la force radiative et des termes sources semi-implicites.

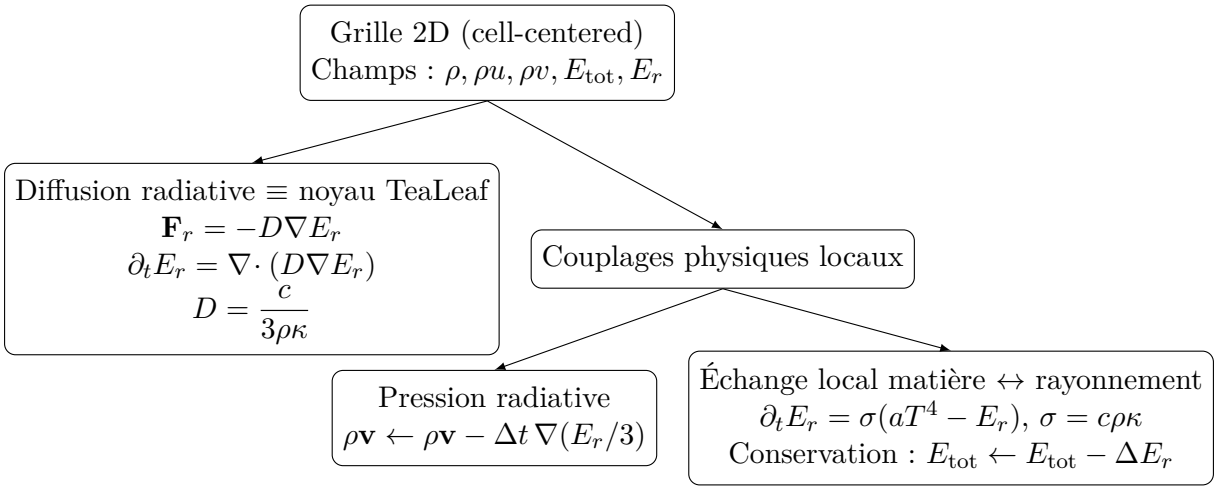


FIGURE 8 – Organisation contextuelle de notre modèle.

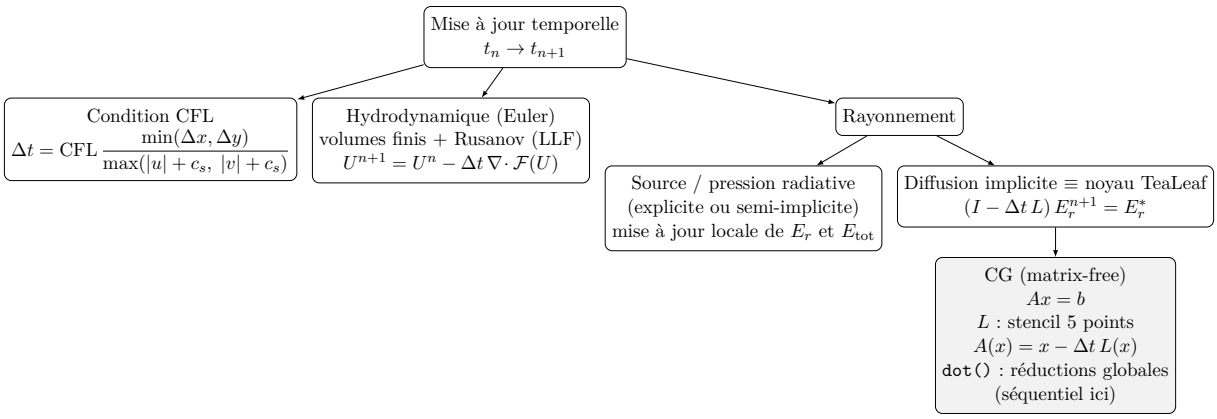


FIGURE 9 – Itération temporelle et noyau diffusif (CG).

9.2 Modularité et Fichiers Fortran

Les fichiers sont organisés de manière modulaire, chaque composant du modèle étant regroupé dans un module Fortran (`.f90`) distinct :

- `config.f90` : paramètres globaux (dimensions $n_x \times n_y$, pas spatiaux,constante CFL, coefficients physiques,paramètres du solveur).
- `hydro.f90` : résolution explicite des équations d'Euler via un schéma de type Rusanov (volumes finis).
- `radiation.f90` : traitement du rayonnement, incluant les termes sources et la diffusion implicite résolue par le Gradient Conjugué.
- `main.f90` : boucle temporelle(calcul de Δt, appel successif aux modules hydrodynamique puis rayonnement).
- `io.f90` : gestion des sorties de simulation (traces d'exécution / logs).

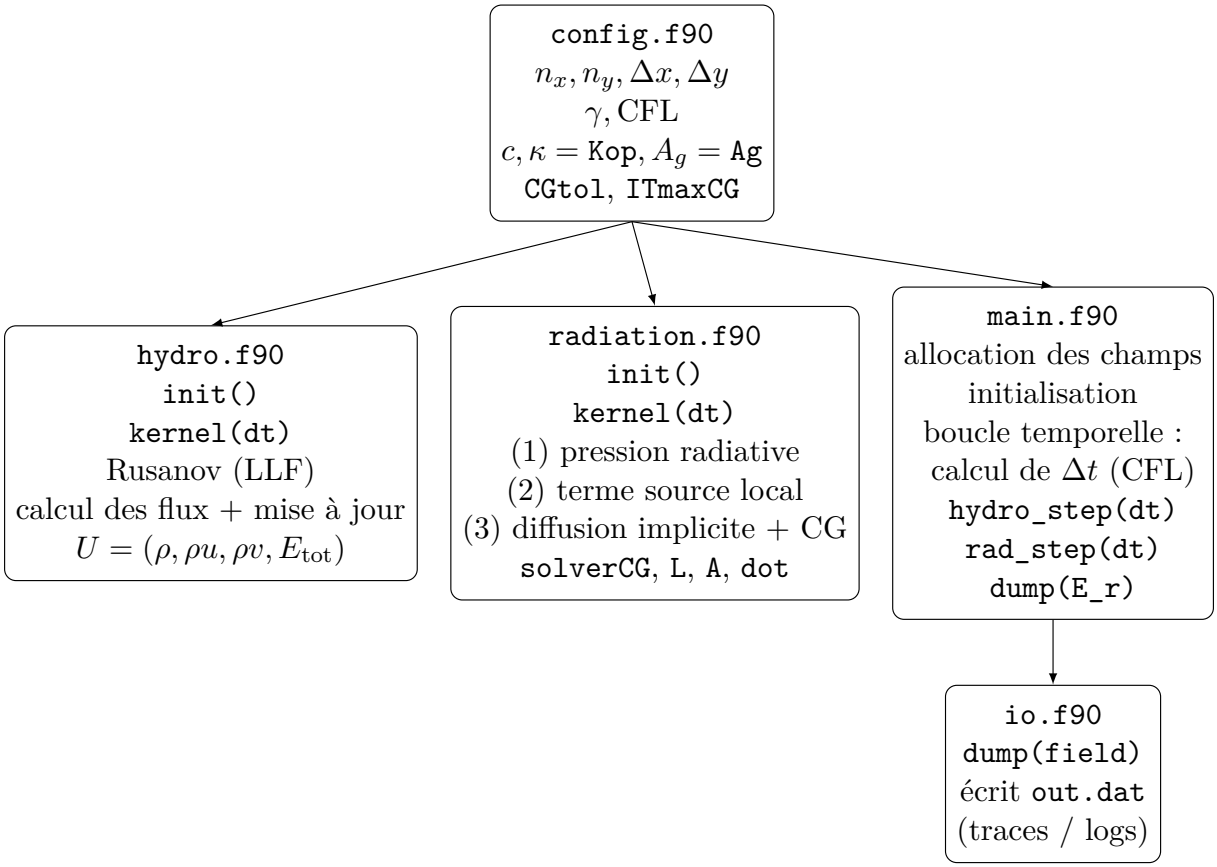


FIGURE 10 – Organisation modulaire du code Fortran.

Visualisation de la diffusion

Voici les résultats obtenus après exécution du code. Les états initiaux sont définis dans `config.f90`, en respectant des ordres de grandeur physiquement cohérents et compatibles avec les domaines de résolution du modèle. Cependant, afin de rendre le phénomène de diffusion plus lisible, nous avons volontairement ajusté certains paramètres.
En pratique, l'exécution en Fortran étant rapide (notamment grâce à la vectorisation im-

plicite de certaines boucles), une configuration réaliste conduit à une homogénéisation très rapide du champ. Il devient alors difficile d'observer visuellement la propagation progressive de la chaleur sur la grille 2D. Nous avons donc choisi des paramètres volontairement exagérés (vitesse initiale, pas de temps maximal, etc.) afin de ralentir la dynamique et de rendre l'évolution du champ plus visible.

L'objectif était d'observer clairement la diffusion dans une direction privilégiée, et de vérifier que la propagation suivait un gradient cohérent jusqu'à homogénéisation progressive du champ.

Afin d'illustrer qualitativement cette propagation thermique, un script `./plot`, basé sur `gnuplot`, lit les valeurs $E_{r,i,j}$ écrites dans le fichier `out.dat`. Ce fichier contient les traces numériques du champ $E_r(i,j)$ pour chaque cellule du domaine, produites à chaque pas de temps. Ces données permettent de générer une visualisation dynamique de l'évolution du champ.

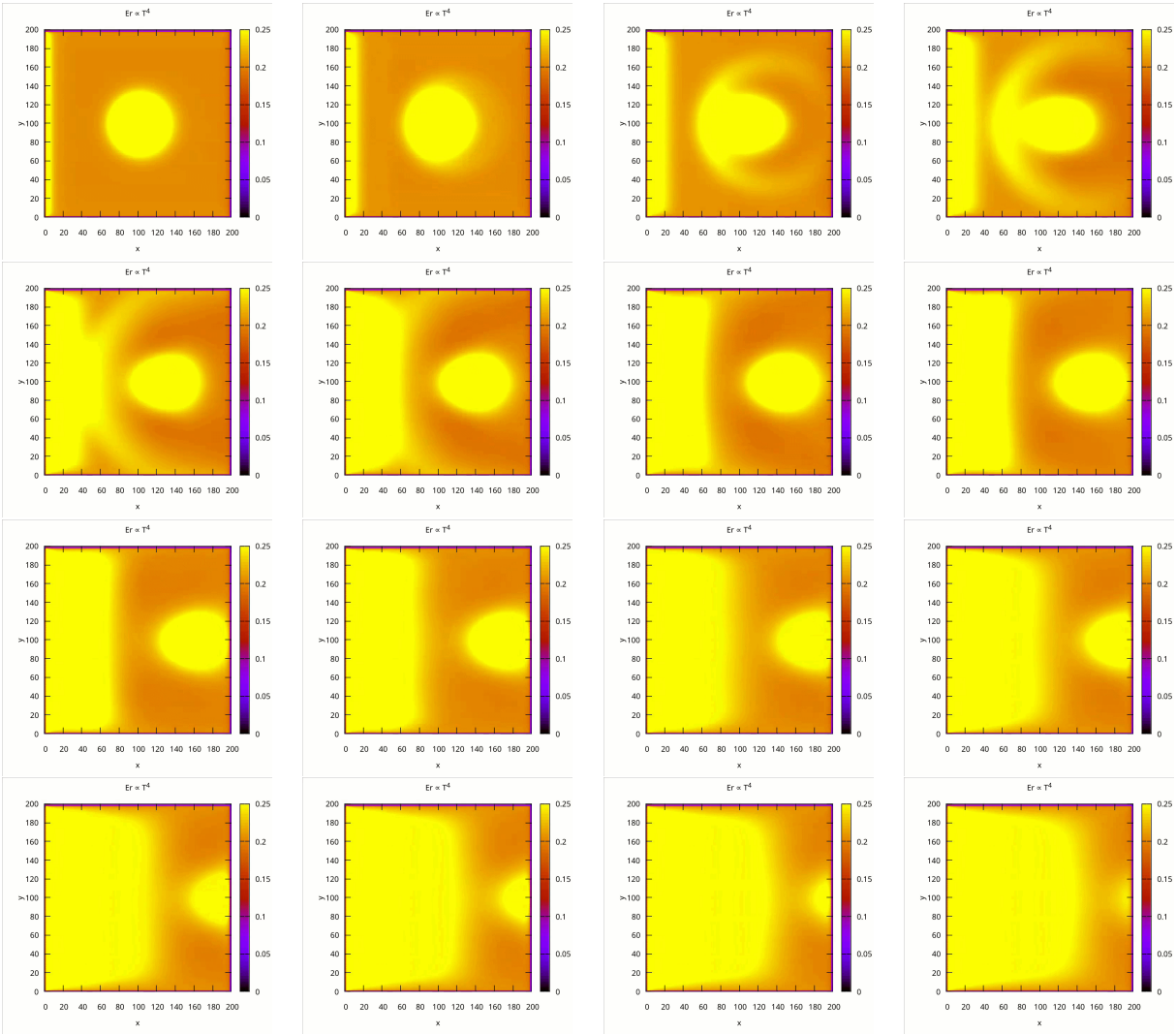


FIGURE 11 – Diffusion radiative : Évolution de $E_r(x,y,t) \propto T^4$.

Bien que cette implémentation ne vise pas les performances d'une mini-application optimisée, elle constitue une approche volontairement naïve, inspirée de l'étude de TeaLeaf.

Elle nous a néanmoins permis de comprendre en profondeur :

- la structure mathématique d'une équation de diffusion ;
- le passage d'une formulation conservative continue à une discrétisation locale en stencil ;
- la construction d'un opérateur implicite ;
- l'intégration concrète d'un solveur CG dans un contexte multiphysique.

Ainsi, au-delà de l'aspect performance, cette implémentation représente une étape importante dans la compréhension des modèles diffusifs et de leur mise en oeuvre numérique.

10 Conclusion

Ce projet autour de la mini-application *TeaLeaf* nous a permis d'explorer concrètement les liens entre modèle mathématique, schéma numérique et contraintes matérielles en calcul haute performance.

Dans un premier temps, l'étude théorique de TeaLeaf et de l'article associé nous a permis de comprendre le rôle central des solveurs itératifs creux dans les simulations physiques modernes. La résolution implicite de l'équation de diffusion conduit à un système linéaire symétrique défini positif, naturellement adapté aux méthodes de Krylov telles que le Gradient Conjugué. L'analyse des variantes préconditionnées (PPCG/CPPCG) met en évidence un enjeu fondamental en HPC : réduire les communications globales tout en conservant la robustesse numérique.

La réimplémentation 1D puis 2D du modèle nous a permis d'appliquer ces principes dans un cadre maîtrisé. Bien que nous ne soyons pas physiciens, ce travail nous a permis de comprendre les mécanismes essentiels de la diffusion implicite, la structure d'un opérateur Laplacien discret, ainsi que l'impact du conditionnement sur la convergence. Cette approche progressive, nourrie par les enseignements du cours MPNA, nous a permis de passer de l'équation mathématique à l'implémentation effective d'un solveur dans un code scientifique.

Notre modèle multiphysique simplifié (hydrodynamique + rayonnement) reste volontairement naïf et comporte plusieurs simplifications importantes : hypothèses physiques réduites, absence de validation expérimentale, comportements non calibrés par rapport à des données réelles. Cependant, l'objectif n'était pas la fidélité physique complète, mais la compréhension algorithmique et architecturale. Cette simulation nous a servi de laboratoire pour expérimenter l'intégration d'un solveur implicite dans une boucle temporelle complète, avec gestion mémoire, modularité Fortran et organisation logicielle structurée.

L'analyse de performance a constitué un second axe majeur du projet. Les expériences de strong et weak scaling ont montré que, sur CPU, l'application devient rapidement *memory-bound*. Malgré l'optimisation séquentielle et l'usage d'un modèle hybride MPI/OpenMP, la saturation du bus mémoire limite fortement le gain au-delà de quelques coeurs. Ce constat illustre une réalité fondamentale du HPC moderne : l'optimisation du calcul pur ne suffit plus lorsque la bande passante mémoire devient le facteur limitant.

L'implémentation GPU (CUDA) a confirmé cette analyse. En exploitant une bande passante mémoire beaucoup plus élevée et un parallélisme massif, le GPU permet de lever en partie ce plafond architectural et d'obtenir des accélérations significatives pour les grandes tailles de problème. Cette comparaison nous a permis de comprendre concrètement l'importance de l'adéquation entre algorithme et architecture.

Au-delà des résultats numériques, ce projet a surtout été formateur sur le plan méthodologique. Il nous a permis de :

- analyser un article scientifique en profondeur et en extraire les choix algorithmiques structurants ;
- comprendre la conception d'une mini-application comme outil d'exploration du design space ;
- implémenter, optimiser et profiler un solveur creux sur différentes architectures ;
- identifier et interpréter un goulot d'étranglement matériel réel ;
- relier modèle mathématique, structure mémoire et performance mesurée.

Ainsi, ce travail dépasse la simple reproduction d'un benchmark. Il constitue une étape importante dans notre compréhension des mécanismes internes des solveurs scientifiques modernes et dans notre formation en tant que futurs ingénieurs et chercheurs en calcul haute performance. Il nous a permis d'acquérir une vision plus mature de l'optimisation, non comme une simple accélération, mais comme une analyse rigoureuse de l'interaction entre algorithme, implémentation et architecture.

Références

[1] Équations d'euler (mécanique des fluides). https://fr.wikipedia.org/wiki/%C3%89quations_d%27Euler. Consulté le 20 février 2026.

[2] Tealeaf : A mini-application for exploring iterative solvers. https://research-information.bris.ac.uk/ws/portalfiles/portal/136789852/tealeaf_accepted.pdf, 2015. UK-MAC Project.

[3] Eleuterio F. Toro and Svetlana A. Tokareva. Rusanov-type schemes for hyperbolic equations : Wave-speed estimates, monotonicity and stability. https://arxiv.org/abs/2412.03522, 2024. arXiv preprint arXiv :2412.03522.